



1

WINDOWS FOUNDATIONAL CONCEPTS



To understand your adversary, first you must understand the battlefield. In our case, the battlefield is the Windows operating system and its underlying components. This chapter will provide an overview of the Windows architecture and introduce the fundamental concepts you'll need in order to understand the more advanced topics covered in later chapters of this book.

Windows Architecture Overview

Windows is a complex operating system (OS) with an equally complex architecture. In this section, I'll introduce some of its key concepts, and as we proceed through the chapter, I'll revisit and elaborate on them.

User and Kernel Modes

The Windows architecture consists of two modes: user and kernel, which are the fundamental components of the OS. A *mode* is a context

in which code runs on the system. *User mode* is what most people think of when using a computer; it consists of the normal, day-to-day software and processes that the user interacts with, such as Microsoft Office programs and web browsers. Conversely, *kernel mode* is reserved for the core OS functions: those responsible for important, low-level tasks like memory management and hardware interaction.

Code running in user mode can't access or interfere with code running in kernel mode; this is an intentional protection mechanism to prevent misconfigured or malicious applications from altering the OS environment. This separation is critical because all code and programs that run in kernel mode share the same memory address space, meaning that a misbehaving program could cause an unintended crash of the entire OS. It also means that if a malicious program were able to execute in kernel mode, it could directly affect the OS.

Figure 1-1 shows the relationship between user mode and kernel mode.

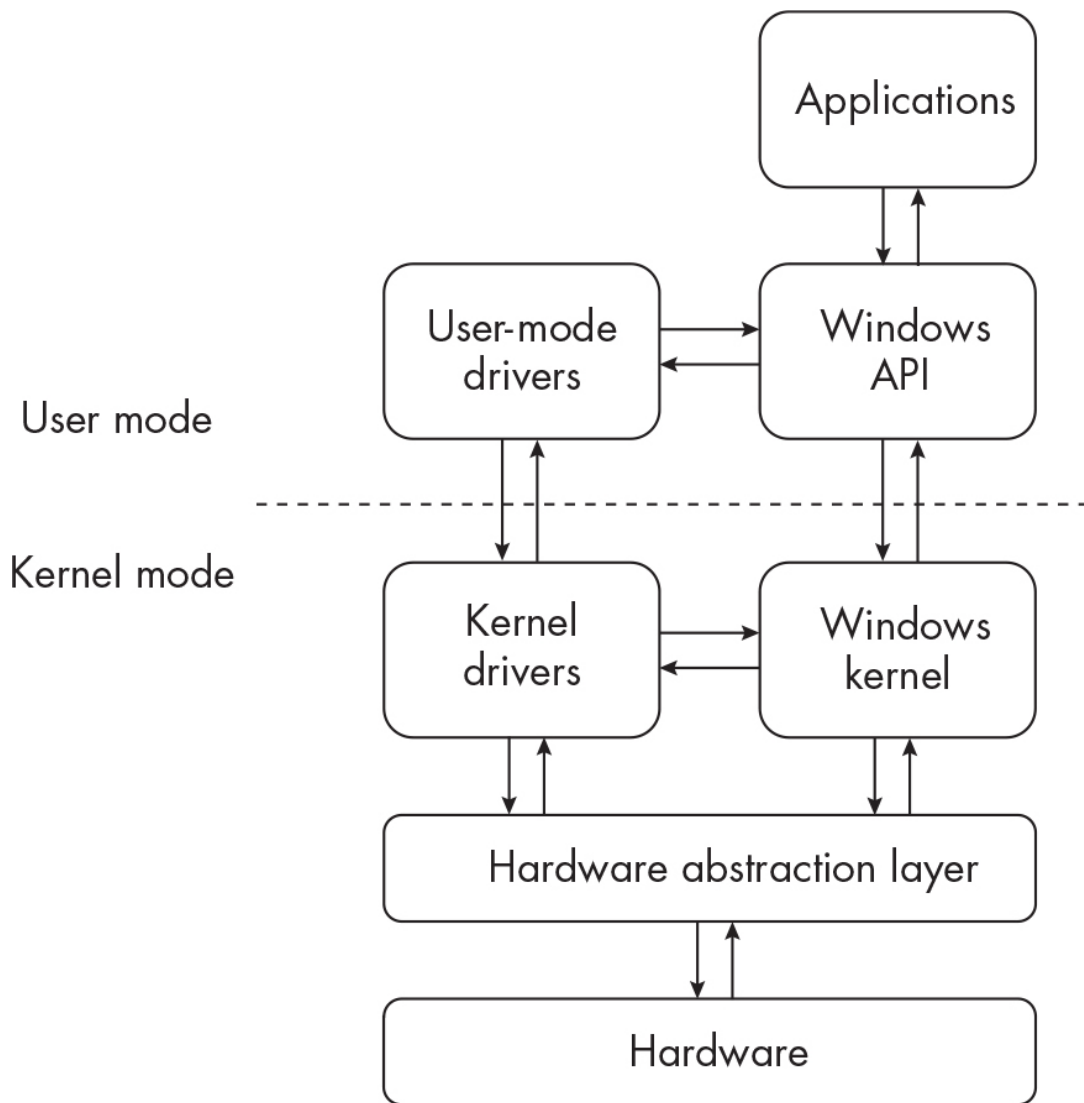


Figure 1-1: The relationship between user mode and kernel mode

Let's take a closer look at the concepts illustrated here:

Applications

These are the software and programs that a user runs.

Windows application programming interface (WinAPI)

This is what applications rely on in order to function. I'll discuss this further in "The Windows API" on [page 6](#).

Drivers

These control various devices on the system and provide an abstraction layer between the devices and the programs that wish to interact with them. There are two kinds of drivers. *Hardware device drivers* take input/output (IO) requests and convert them to hardware IO requests, such as for the mouse and keyboard. *Nonhardware device drivers* control system components such as network interfaces and the filesystem. Some drivers operate in user mode and some in kernel mode.

Hardware abstraction layer

This provides an interface that device drivers can use to communicate with the underlying system hardware. It enables the kernel and higher-level applications to operate independently of the system hardware.

The Windows kernel itself is contained in an executable file called *ntoskrnl.exe*, and it's split into two parts: the kernel and the executive layer. The *kernel* is responsible for fundamental functionality like task synchronization and scheduling; it also provides low-level hardware support, which is essential for the system to run efficiently. The *executive layer* contains critical system services such as the *Memory Manager*, which implements virtual memory functionalities (see “Virtual Memory” on [page 11](#)), and the *Process Manager*, which handles the creation and termination of processes and threads.

Processes and Threads

When an application such as *Excel.exe* or *Calculator.exe* is executed, the Windows Portable Executable loader creates a *process* for that program; the process contains the original program executable and all supporting libraries and code. Each process is assigned its own virtual address space in memory, which is private and isolated to that process. This means that if a process crashes, it does not (or should not) affect other processes or the OS itself.

Each running process in Windows can host one or more threads. A *thread* is simply a series of instructions. The processor can run only one thread at any given time, however. This means that if Process B wishes to execute some code, it must wait for Process A's current thread to complete. If Windows deems Process B to be more important, it can issue an interrupt to Process A's current thread and execute Process B's thread instead. (This is called a *context switch*.) Windows executes context switching so quickly and efficiently that the end user isn't even aware that it's constantly happening.

Windows is a multitasking OS. This means that as the processor executes instructions to run the system and its applications, it efficiently switches between executing code in user mode and in kernel mode. To do this, Windows uses *time slices*, which are atomic segments of time (measured in milliseconds) that are assigned to each thread. A thread is allowed to execute its code, but after its time slice expires, other threads are allowed to run. Because time slices are so small, the system appears to be executing multiple threads and operating multiple programs at the same time.

Objects and Handles

Processes and threads often interact with *objects*, which are instances of a certain type of resource, such as a file, another process or thread, a security token (for user access rights), or even a section of memory. The centralized *object manager* is responsible for tracking all Windows objects, sharing them among processes, and protecting them from unauthorized access.

All objects in Windows are simply data structures, typically stored in kernel memory. Since most applications on Windows run in user space, a process uses a unique identifier known as a *handle* to access an object. Each process may have multiple handles for various objects. Handles are managed in a process's *handle table*, which contains pointers to the objects in kernel memory, as illustrated in **Figure 1-2**.

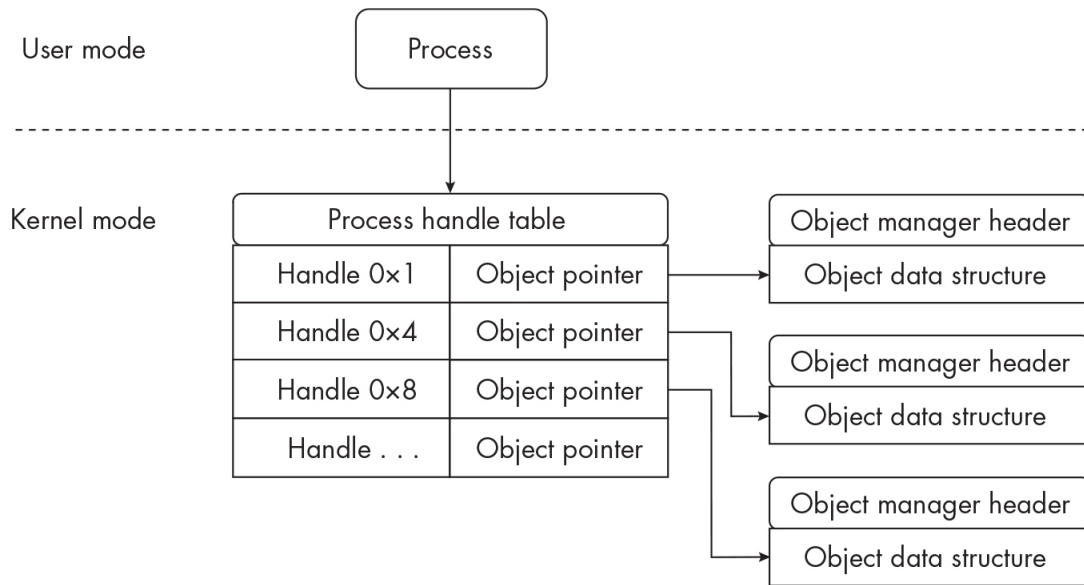


Figure 1-2: A Windows process handle table and object structure

Mutual exclusions, or *mutexes*, are a method of controlling access to objects to prevent potential issues like Process A and Process B both attempting to access and modify a file at exactly the same time, which is known as a *race condition*. Depending on how Windows uses this specific file, this situation could potentially cause data inconsistencies, or worse, crash a process or the OS. To avoid unwanted events such as a race condition, Process A may create a mutex object for that file, locking the file for its own use and preventing Process B from being able to access or write to the file until it is unlocked.

The program's interaction with these objects, and with the OS itself, is managed by the Windows API.

The Windows API

The *Windows application programming interface (WinAPI)* is a shared library of code that is exposed to user-mode applications. When a Windows program runs, WinAPI invokes Windows functions that enable the program to operate as designed within the Windows OS.

WinAPI covers nearly all the functionality that a developer could want to implement in their code: everything from user interface and net-

working capabilities to input devices (mouse, keyboard, and so on) to memory management. For example, if a developer wants to create a new window for their application, they might call the `CreateWindowEx` function. If a program needs access to a hard disk, it might call `GetLogicalDrives` to retrieve a list of the available hard disks.

There's also a lower-level API called the *Windows Native API*, or simply *Native API*. While WinAPI is well documented and designed to be used by developers, the Native API is largely undocumented, at least by Microsoft.

NOTE

Thankfully, the reverse engineering community has gone to great lengths to document the Native API internals. Two great examples are the “ntinternals” project (<http://undocumented.ntinternals.net>) and Geoff Chappell's research (<https://www.geoffchappell.com/studies/windows/win32/ntdll/api/native.htm>).

The Native API is designed to be internal to the OS, but programs can call Native API functions directly if they so choose. In turn, Native API functions call into even lower-level kernel API code residing in *ntoskrnl.exe*. A call into the kernel is known as a *syscall* or *sysenter*. (There are minor technical differences between *syscall* and *sysenter*, but they both have the same objective: allowing user applications access to kernel services. For simplicity's sake, I'll use the term *syscall* in this book.)

Figure 1-3 illustrates the interaction between WinAPI and the Native API.

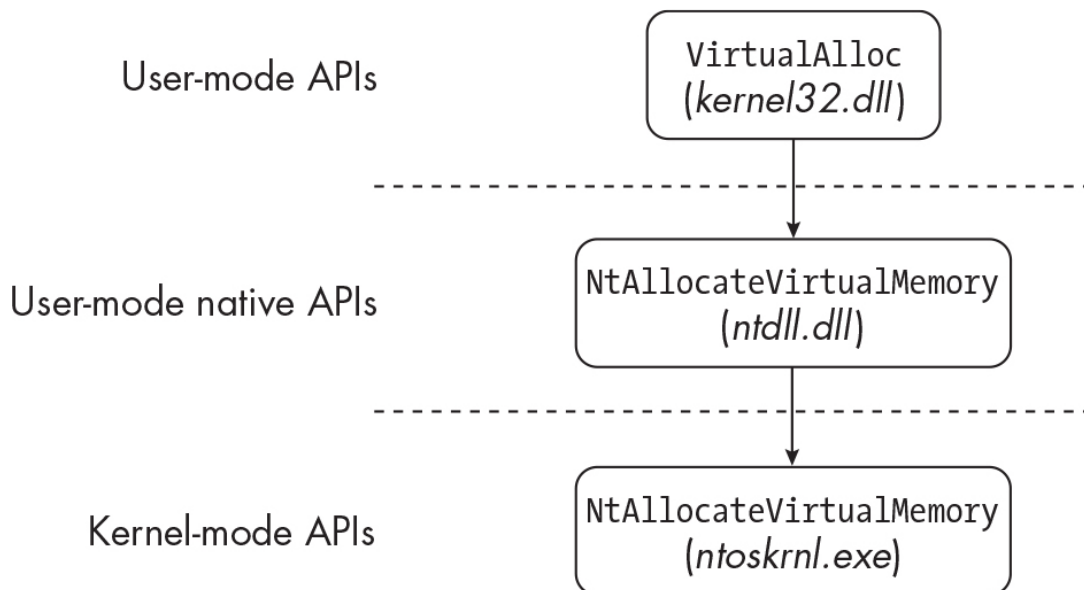


Figure 1-3: User-mode APIs calling into lower-level APIs

At the top, you can see a program calling the WinAPI function `VirtualAlloc`, which in turn calls WinAPI's `VirtualAllocEx` function (not shown in this diagram). This is followed by a call to the Native API's `NtAllocateVirtualMemory` function. Finally, the program invokes the `NtAllocateVirtualMemory` function inside `ntoskrnl.exe`. This complex chain of API calls is very common in Windows and allows for code to be developed without the developer needing to understand all of the internals of Windows and the lower-level APIs.

As you may have noticed, WinAPI functions often have one or more suffixes, such as `Ex`, `A`, or `w`. Generally speaking, the `Ex` suffix is Microsoft's way of designating a newer, extended (with more features) version of an older function. For example, the `CreateWindowExA` function is the newer, extended version of the `CreateWindowA` function. The `A` suffix indicates that the function uses ANSI format inputs and outputs, while functions with a `w` suffix use Unicode inputs and outputs. The usage of ANSI versus Unicode doesn't matter that often, so I won't be covering that topic further and I generally won't include the `A` or `w` in the function name. (For example, I'll refer to the `CreateWindowA` function as simply `CreateWindow`.) For `Ex` functions, however, I'll always include the suffix.

Windows Native API functions typically have the prefix `Nt` or `Zw`; examples include `NtCreateProcess` and `ZwNotifyChangeKey`. Each function in the API typically has both an `Nt` and a `Zw` version. `Zw` functions are most often used by drivers and other lower-level system software but are largely interchangeable with `Nt` functions.

You may also have noticed the filenames *kernel32.dll* and *ntdll.dll* in **Figure 1-3**. These refer to *dynamic link libraries (DLLs)*, a collection of resources that developers can import into their program to make use of existing code from Microsoft or third-party developers. While Windows programs can import these libraries, DLL files themselves export functions to the program that imported them. For example, if a developer wishes to access a hard disk on the Windows system, they can import the *kernel32.dll* library, which exports (provides) the `GetLogicalDrives` function they need. Some of the DLLs used often in Windows programs are as follows:

kernel32.dll This is one of the primary DLLs required for Windows programs to run, and it contains many of the fundamental user-mode functions.

user32.dll This DLL provides the graphical user interface (GUI) functions required for Windows programs.

Winhttp.dll Also known as the “Windows HTTP Interface,” this DLL provides internet connection functionalities to Windows programs.

ntdll.dll This critical DLL contains functions for synchronization, threading, and other system tasks; it also communicates with the kernel.

NOTE

If you're analyzing code with functions exported from `kernel32.dll` (as many functions are), you might notice that when a process executes a

kernel32.dll function, there's an immediate jump to another DLL, kernelbase.dll. Introduced in Windows 7, kernelbase.dll allows for backward compatibility between older and newer Windows versions. Most kernel32.dll function calls simply jump to kernelbase.dll, which contains the function's actual code. For example, the function `WriteFile` (exported from kernel32.dll), once invoked, will immediately jump to kernelbase.dll, where its code actually resides. For simplicity's sake, I'll use kernel32.dll and kernelbase.dll synonymously throughout this book.

While legitimate Windows programs heavily rely on the underlying WinAPI and Native API to function, illegitimate software (that is, malware) also uses these functionalities, as you'll see in later chapters. For now, let's take an in-depth look at processes.

Process Internals

Processes are quite complex data structures that point to additional data structures. This network provides the underlying information that Windows relies on to efficiently manage and coordinate the many processes running on a system at any point in time.

EPROCESS Structures

Even though most processes exposed to the end user run in user mode, they're represented in kernel address space as objects called *EPROCESS* structures. Each process has its own *EPROCESS* structure that contains pointers to elements like the list of handles the process has open, as well as its *Process Environment Block (PEB)*, which is a structure that contains vital information about the process. *EPROCESS* structures consist of a doubly linked list; that is, they form a chain in which each structure links to the previous and subsequent structures. *EPROCESS* structure members called *forward links (flinks)* are pointers to the next *EPROCESS* structure in the chain, while *backward links (blinks)* point to the previous one. **Figure 1-4** illustrates a simplified version of this chain.

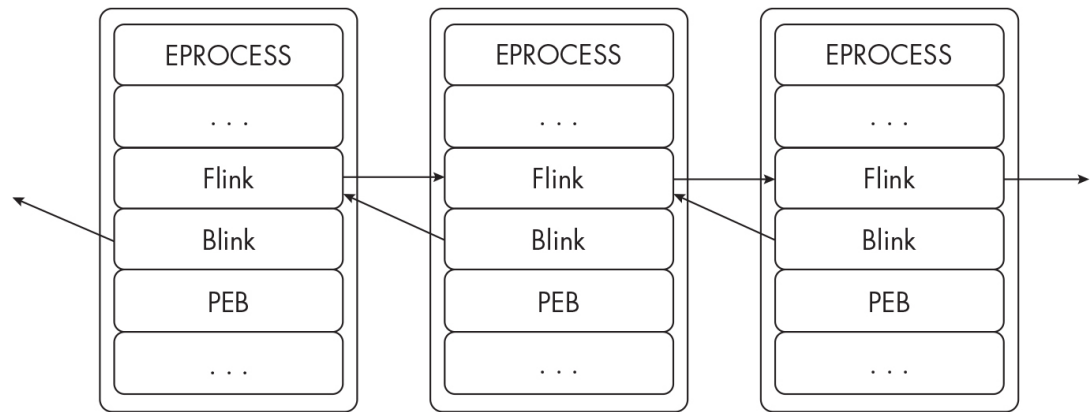


Figure 1-4: An EPROCESS structure’s doubly linked list chain

The EPROCESS structure contains many other data elements and pointers that are mostly outside the scope of this book. The key take-away here is that each user-mode process in Windows is linked to an EPROCESS structure running in kernel mode. This will be important later on, when we discuss topics like direct kernel object manipulation (DKOM) in **Part IV**.

Process Environment Blocks

The PEB memory structure contains information about a running process that the kernel needs to communicate with that process, as well as information for interprocess communication. Each running process has its own PEB that’s stored in user-mode address space inside that process memory. **Table 1-1** lists some of the important data the PEB structure contains. The offset of each structure member is shown for both x86 and x64 architectures.

Table 1-1: PEB Structure Offsets and Data

Offset (x86)	Offset (x64)	Data
0x002	0x002	Stores the BeingDebugged value, which indicates whether the process is running under the con-

Offset (x86)	Offset (x64)	Data
		text of a debugger. (This will be important in Chapter 10 .)
0x008	0x10	Stores the base address of the process executable in memory.
0x00C	0x18	Stores information on the modules and libraries the process has loaded.
0x018	0x30	Stores information about the process's memory heap.
0x064	0xB8	Stores the <code>NumberOfProcessors</code> value, which indicates the number of processors the system has.

You don't need to memorize all the elements of a PEB, but it's important to have a basic understanding of it because I'll be referencing it throughout the book.

Thread Environment Blocks

A *Thread Environment Block (TEB)*, sometimes referred to as the *Thread Information Block (TIB)*, contains information for a process's running threads. Much like the PEB, it's simply a data structure that stores critical information for each thread and is stored in the memory address space of the process that owns the thread. [Table 1-2](#) lists some of the most interesting elements in the TEB.

Table 1-2: TEB Structure Offsets and Data

Offset (x86)	Offset (x64)	Data
FS: [0x00]	GS: [0x00]	Stores the current structured exception handler (SEH) frame. (SEH will be covered in more detail in Chapters 10 and 11 .)
FS: [0x04]	GS: [0x08]	Points to the base of the thread's stack (see the next section).
FS: [0x18]	GS: [0x30]	Points to the TEB itself.
FS: [0x20]	GS: [0x40]	Stores the <i>process ID (PID)</i> of the thread's owning process.
FS: [0x24]	GS: [0x48]	Stores the <i>thread ID (TID)</i> of the current thread.
FS: [0x30]	GS: [0x60]	Points to the PEB of the thread's owning process.
FS: [0xE10]	GS: [0x1480]	Points to thread-local storage (TLS) information.

NOTE

Thread-local storage (TLS) *is used to store variables and other information across different threads. We'll talk about how malware can abuse TLS to stealthily execute malicious code in [Chapter 11](#).*

Stacks and Heaps

A process can have multiple active threads, each of which has its own memory stack. The *stack* is where the thread stores temporary data such as variables, pointers, and other objects that will inevitably be destroyed once the thread completes execution and is terminated. Since the stack is so volatile and temporary, programs sometimes need a more “permanent” solution for data storage. This is where heaps come in.

A *heap* is a variably sized region of memory that a program dynamically allocates at runtime. Heaps are often used to store objects and data structures that are too large for the stack. They’re also used to store global variables and data that persist and can be used by multiple functions in the same program. It’s important to note that while the stack is largely managed by the OS, heaps are managed by the program itself. If a program doesn’t implement heap memory management techniques well, it can cause stability issues.

Virtual Memory

Each process running in Windows has a number of virtual memory regions assigned to it that are mapped to physical memory (or RAM). It’s important to understand the distinction between virtual and physical memory. *Physical memory* is the real, tangible hardware memory that is installed in your system. If you have a computer with 8GB of RAM, then you have 8GB of physical memory that all processes running on your system share. This poses a problem in that any process can interfere with any other process on the system (either inadvertently, such as in the case of a crash, or purposefully), with unwanted side effects. This is where virtual memory comes in.

Virtual memory is a sort of barrier between physical memory and process memory address space. When a process is started, it’s assigned an allotment of virtual memory that is mapped to physical memory

via the *page table*. The page table keeps track of where different segments of virtual memory are physically located in RAM. **Figure 1-5** illustrates the relationship between virtual memory, the page table, and physical memory.

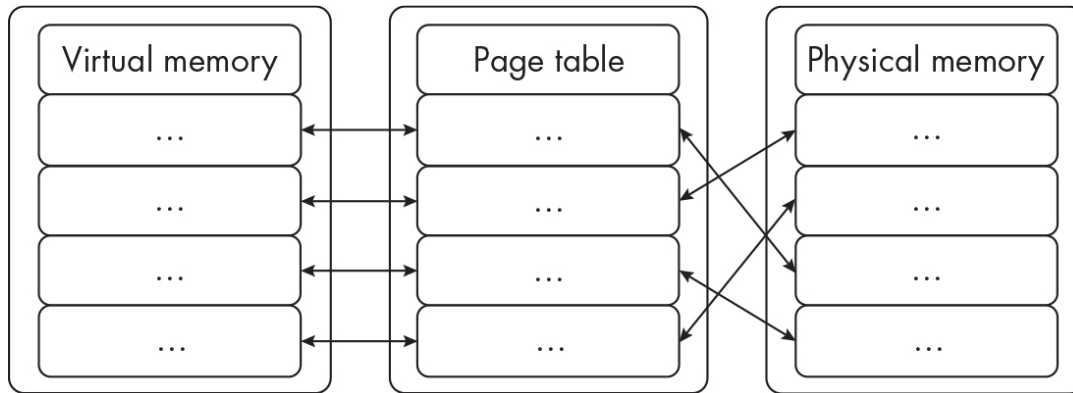


Figure 1-5: Virtual memory mapped to physical memory via the page table

Each block containing an ellipsis (...) represents a memory address range, or region. Each region of memory is mapped to the page table.

It's quite possible for a system to have less RAM than what is required by all its running processes. To help manage this, virtual memory can be *paged out*, meaning that it will be temporarily stored on the hard disk when unused. If a process requires access to that virtual memory region again, the memory can be read from disk and remapped to physical memory.

You can view the virtual memory of a process using a process analysis tool such as Process Hacker (<https://processhacker.sourceforge.io>). To do this, start a program (such as *Calculator.exe*), open Process Hacker, and double-click the process you wish to explore. **Figure 1-6** shows the Memory tab of a process in Process Hacker.

Base address	Type	Size	Protection	Use	Total WS	Private WS
> 0x1e8816f0000	Mapped	796 kB	R	C:\Windows\System32\locale.nls	680 kB	
> 0x1e8817c0000	Mapped	32 kB	R		16 kB	
> 0x1e8817d0000	Private	4 kB	RW		4 kB	4
> 0x1e8817e0000	Private	4 kB	RW		4 kB	4
> 0x1e8817f0000	Private	44 kB	RW	Heap (ID 4)	4 kB	4
> 0x1e881800000	Private	2,048 kB	RW		1,580 kB	1,580
> 0x1e881a00000	Mapped	280 kB	WC		204 kB	
> 0x1e881a50000	Mapped	2,048 kB	R		24 kB	
> 0x1e881c50000	Mapped	1,540 kB	R		28 kB	
> 0x1e881de0000	Mapped	20,484 kB	R		52 kB	
> 0x1e8831f0000	Mapped	3,292 kB	R	C:\Windows\Globalization\Sorting\So...	104 kB	
> 0x1e883530000	Image	632 kB	WCX	C:\Windows\System32\WinMetadat...	88 kB	
> 0x1e8835d0000	Private	8,192 kB	NA		4 kB	4

Figure 1-6: Viewing the virtual memory of a process in Process Hacker

The Base address column contains the base memory address of each virtual memory region assigned to *Calculator.exe*. The Type column contains the memory type for each region, the Size column gives the allocation size of each region, and the Protection column lists the protection status of the region.

Each virtual memory region is typically assigned one of three common memory types:

- *Image (IMG)* memory usually contains executable files or libraries that have been mapped into memory via the standard Windows loader mechanism (described shortly).
- *Mapped (MAP)* memory often contains either files that have been mapped into memory from the disk or other data used by the application running inside the process.
- *Private (PRV)* memory is typically allocated via `VirtualAlloc` and similar memory allocation functions.

Furthermore, each memory region can be either committed or reserved. *Committed* regions are being actively used and have been mapped to physical memory. *Reserved* regions are reserved for the process but aren't in active use and haven't yet been mapped to RAM.

Now let's dig into the details of how Windows executables (more specifically, Portable Executable files) work internally.

The PE File Format

Microsoft created the *PE (Portable Executable)* file format for executable files that run inside the Windows OS. The PE file format contains everything the Windows PE loader needs to execute the embedded code. Understanding the PE format is critical to understanding how malware works, so in this section we'll look in depth at the structures the PE format comprises.

NOTE

I'll refer to both x86 and x64 PE files as PE files. In reality, however, x64 has its own version of the PE format called PE32+. Since PE32+ differs only slightly from its x86 equivalent, I won't cover it separately.

Headers and Sections

The PE file format contains several *headers*: metadata or other information at the top of a file, telling the OS and other software what to do with its contents. The *DOS header* contains information required by MS-DOS and very early versions of Windows, and it mostly exists for legacy reasons. The *PE header* contains information used by the Windows PE loader, such as the CPU architecture the executable was compiled to run on and metadata like the file's compilation timestamp. The PE header also includes the *optional header*, which indicates important information such as the PE's base memory address (the memory address at which the PE will be mapped into memory), the size of the code inside the executable, and the target OS that the executable will run on. The "optional" header is in fact no longer optional in modern Windows systems.

The PE file also includes the *section header*, which contains metadata related to each of the file's sections (where the actual file contents are stored), such as the section's size, address, and other characteristics. Finally, most PE files contain at least a few of the following sections:

.text The file's main executable code

.rdata Read-only data, such as static variables and constants

.bss Uninitialized data, such as variables that haven't been assigned a value yet

.data Variables not embedded in the *.rdata* and *.bss* sections, such as global variables

.rsrc Assets that will be loaded by the executable at runtime, such as images, fonts, and other supporting files

.idata The imports address table (see the next section)

.edata The exports address table (see the next section)

Imports and Exports

The *.idata* and *.edata* sections are two of the most important components of a PE file. The *.idata* section contains information about the functions that the PE file will import at runtime. Once the PE file is executed, the program will load the libraries and functions referenced here into memory and build its *import address table (IAT)*, which maps the imported Windows API functions to their addresses in memory.

The *.edata* section contains information about the functions that the PE file exports to other programs, which they can then import and load into memory for their own use. It's common for DLL executable

files, for example, to contain a list of exported functions. As with imports, exports have their own table called the *export address table*.

NOTE

In practice, however, both the .edata and .idata sections are often contained in the .rdata section.

Now let's take a look at the way Windows loads PE files into memory. The program execution process is a fundamental concept for Windows malware analysis and will tie together all of the concepts you've seen so far.

The Windows PE Loading Process

When you launch an executable file like Firefox in Windows, here's what happens:

1. Windows creates a new EPROCESS data structure for the Firefox program and assigns a new process ID.
2. Windows initializes the virtual memory required for the process, creates the PEB structure, and loads two libraries that nearly all Windows processes require: *ntdll.dll* and *kernel32.dll*. It then prepares to load Firefox's PE file by initializing the PE loader.
3. The PE loader parses the DOS, PE, and optional headers of the PE file to gather all information required to successfully execute the file.
4. The PE loader parses the section header to prepare for mapping these sections into memory. The PE loader maps each section into virtual memory within the new process.
5. The PE loader loads all libraries referenced in the imports (usually *.idata* or *.rdata*) section and resolves all addresses for the functions required. All addresses are then stored in the IAT inside the process.

6. A new thread is created inside the current process, and the loader executes the first bytes of code in the executable (usually in the `.text` section).

Figure 1-7 illustrates a PE file being loaded and mapped into virtual memory.

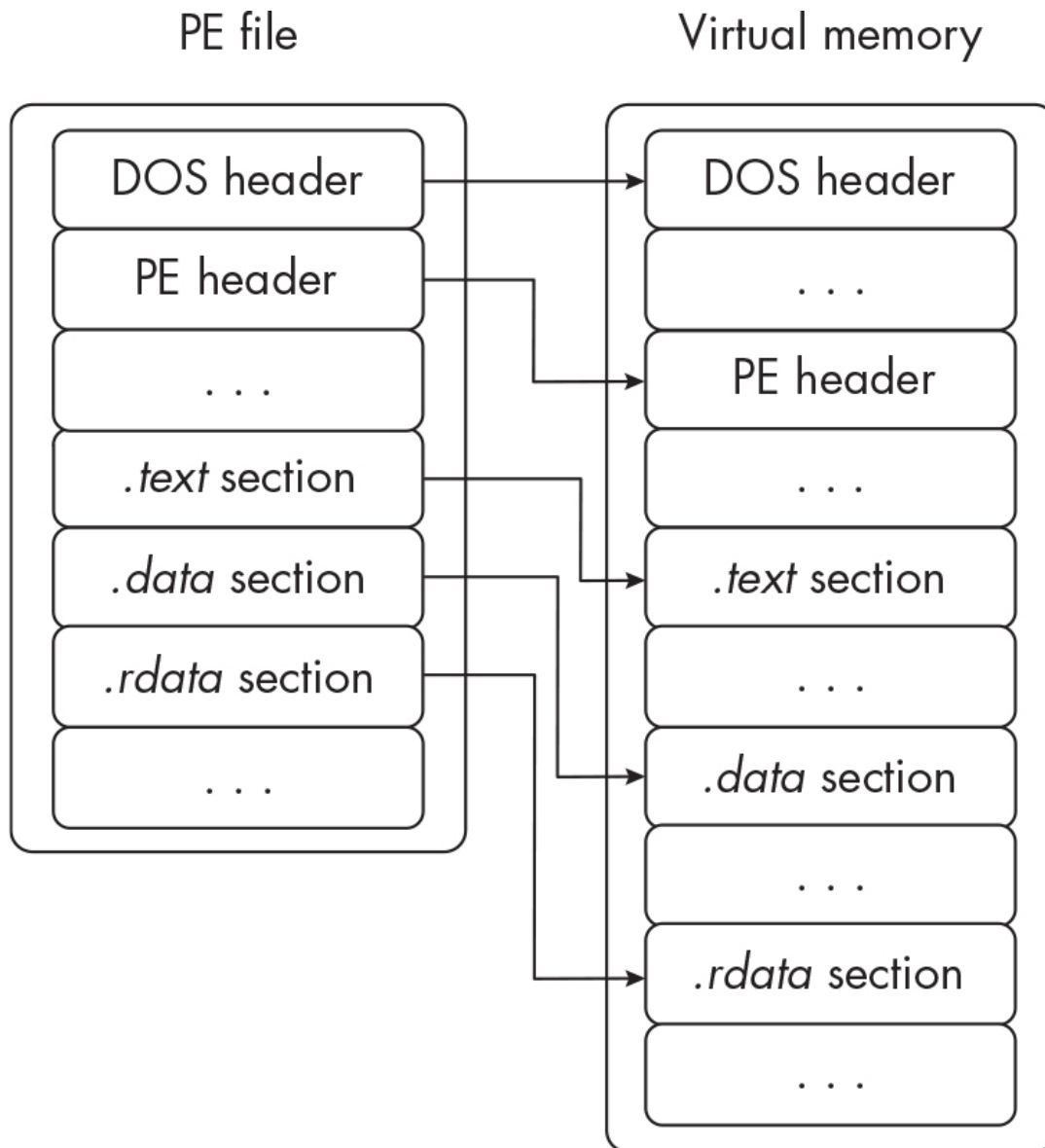


Figure 1-7: Loading and mapping a PE file into virtual memory inside a process

Each section in the PE file is individually mapped into memory, but it appears expanded in virtual memory as there are often regions of memory between each section.

ADDRESS SPACE LAYOUT RANDOMIZATION

Address space layout randomization (ASLR) is a technique used to thwart attacks such as memory corruption. *Memory corruption* can occur when a program controlled by an attacker writes malicious code into a memory region that would normally be outside the program's scope (such as another process). This can cause the program or OS to crash, or worse, to grant the attacker control of the system. Memory corruption attacks take advantage of the fact that Windows loads executables and libraries into memory at foreseeable addresses; that is, attackers already know where the Windows loader will map executables and libraries into memory and will try to corrupt this memory to run malicious code.

ASLR randomizes the locations in memory where these executables and libraries are loaded, making the attacker's life a lot more difficult. ASLR is an in-depth topic, and much research has been dedicated to both implementing and bypassing it. I will revisit this concept later, but a basic understanding of ASLR's functionality is all that's required for this book.

The Registry

The final Windows concept to discuss is the *registry*, which is simply a database that the OS and other installed applications use to store configurations and settings. The registry stores data hierarchically, with several primary root keys or *hives*, each containing additional keys (which you can think of as directories), each of which in turn stores more keys or values.

Values are the actual configuration for the settings. For example, the root key `HKEY_CURRENT_USER` contains a subkey called `Control Panel`, which itself contains a subkey called `Mouse`. `Mouse` contains multiple values, such as `MouseSpeed`, which stores the mouse speed configuration for the currently logged-in user. This registry key path can be expressed just

like a file or directory path in Windows:

HKEY_CURRENT_USER\Control Panel\Mouse\MouseSpeed.

These are five of the most important hives in the registry:

HKEY_LOCAL_MACHINE (HKLM)

Values specific to the system, such as low-level OS and hardware configurations, security policy and account settings, and settings for various software installed.

HKEY_CURRENT_USER (HKCU)

Values related to user settings and system configurations, like sound, mouse, keyboard, network, and printer settings.

HKEY_USERS (HKU)

Values related to user settings for each user with an account on the system. Under this root key, there are several other subkeys starting with S (S-1-5-20, for example). Each subkey represents the identifier for a user account on the system and stores configuration information for each user.

HKEY_CURRENT_CONFIG (HKCC)

Pointers to the HKEY_LOCAL_MACHINE key for the hardware profile currently in use by the logged-in user. This key is less important for our purposes, as everything of value is stored in the HKEY_LOCAL_MACHINE key.

HKEY_CLASSES_ROOT (HKCR)

Information related to registered applications, such as file associations, which map file types to the application that can process them. (For example, .doc files should open in Microsoft Word.)

Registry hives are stored on the hard disk as files. When Windows boots up, these files are loaded into memory, and the registry is built. Any changes to the registry after the system boots up are stored in memory and not directly on disk. This is why some malware is able to store malicious code and configurations in the registry without necessarily touching the disk.

Finally, Windows has a built-in utility called the Registry Editor (*Regedit*), shown in **Figure 1-8**, that all malware analysts should be intimately familiar with.

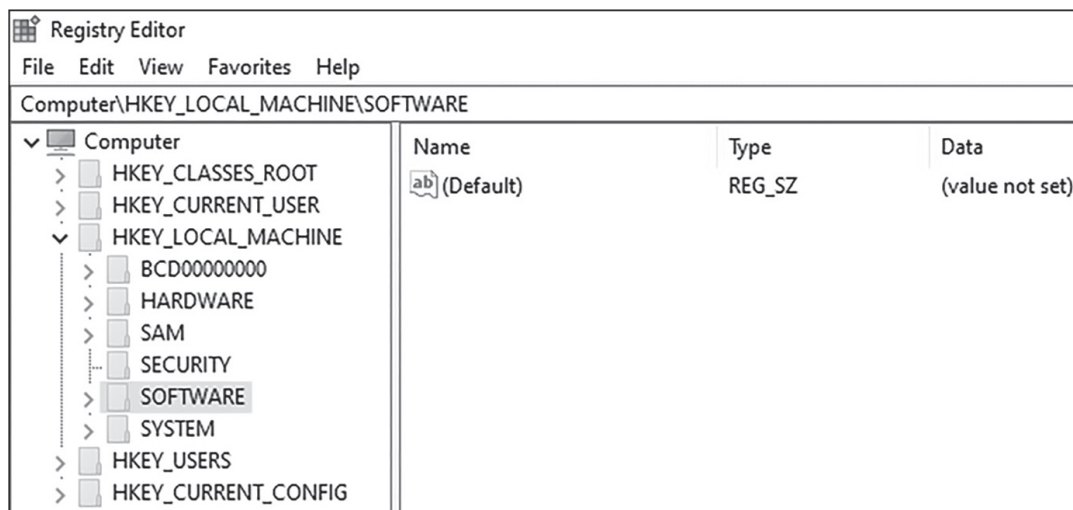


Figure 1-8: The Registry Editor tool in Windows

Regedit allows you to inspect and modify each registry key and value on the system, which is useful for understanding how the registry works. Regedit can also be useful for investigating how malware may have altered this data.

Summary

This chapter discussed some of the fundamental concepts, objects, and structures used by the Windows operating system and explored the architecture of Windows at a high level. We'll return to these concepts throughout the book as we look at ways in which attackers can exploit some of these features to execute malicious code while evading de-

fenses. In the next chapter, we'll walk through the basics of the malware triage and behavioral- analysis process to lay the groundwork for our investigations in later chapters.